Claude Certified Architect - Foundations

Practice Question Bank · Set A · 60 scenario-based questions

This set contains 60 original, scenario-based multiple-choice questions covering all five domains of the Claude Certified Architect - Foundations exam. Questions are framed around production scenarios such as a customer support resolution agent, a multi-agent research system, Claude Code in team and CI workflows, and structured data extraction. Question counts per domain follow the published exam weighting.

Domain	Exam weight	Questions in this set	Question numbers
D1. Agentic Architecture & Orchestration	27%	16	1-16
D2. Tool Design & MCP Integration	18%	11	17-27
D3. Claude Code Configuration & Workflows	20%	12	28-39
D4. Prompt Engineering & Structured Output	20%	12	40-51
D5. Context Management & Reliability	15%	9	52-60

How to use this set

- Treat it as a mock exam: 60 questions, 120 minutes, closed book, no AI help.
- Each question has one best answer. Several options are often partly true; choose the one that best addresses the root cause in the scenario.
- Mark your answers, then check the Answer Key and read every explanation, including the ones you got right.
- Record your score per domain on the score sheet at the end. Aim for at least 75% in every domain, not just overall.

Note: these are original practice questions written to match the publicly described exam blueprint (domains, weights, scenario style). They are not actual exam questions. Weights and format are as reported in October 2026; check Anthropic's official exam guide before you sit the exam.

Domain 1: Agentic Architecture & Orchestration

Exam weight 27% · 16 questions

QUESTION 1 · SCENARIO: CUSTOMER SUPPORT AGENT

Your support agent built on the Claude API decides when to stop looping by checking whether the assistant's text contains the word "DONE". In production it sometimes stops while a tool call is still pending, and sometimes keeps looping after it has fully answered. What is the most reliable fix?

A. Replace the keyword check with a regular expression that matches several natural-language completion phrases such as "all done" or "resolved".

B. Set a fixed maximum of three iterations as the primary stop condition, since most support cases resolve within three tool calls.

C. Continue the loop while `stop_reason` is `"tool_use"`, appending each tool result to the conversation, and stop when `stop_reason` is `"end_turn"`.

D. Instruct Claude in the system prompt to end every response with a JSON object `{"done": true|false}` and stop when it is true.

QUESTION 2 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A coordinator agent delegates to a web-search subagent and then to a synthesis subagent. The final reports are well written but rarely cite any of the sources the search subagent found. Logs show the search subagent returned rich results with URLs. What is the most likely cause?

A. The synthesis subagent's `max_tokens` is too low, so citations are truncated from the end of its output.

B. The synthesis subagent needs its own web-search tool so it can independently re-find the sources it is writing about.

C. Subagent outputs are cached by the SDK, so the synthesis subagent is reading a stale copy of the search results.

D. Subagents do not inherit the coordinator's conversation, so the coordinator must pass the search results and their source metadata explicitly in the synthesis subagent's prompt.

QUESTION 3 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your coordinator researches five independent sub-topics by invoking a subagent for each. End-to-end latency is five times that of a single subagent run because they execute one after another. What change most directly reduces latency?

A. Merge the five sub-topics into one subagent prompt so a single subagent handles all of them in one context.

B. Have the coordinator emit all five subagent (Task) tool calls in a single response so the subagents run in parallel.

C. Reduce each subagent's allowed tools to one so that each run completes faster.

D. Replace the coordinator with five separate SDK processes that each write directly to the final report file.

QUESTION 4 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

You configure a coordinator with the Claude Agent SDK, define three subagents with descriptions and prompts, and set `allowedTools` to `["WebSearch", "Read"]`. The coordinator never delegates; it does all of the work itself. What is the most likely reason?

A. Subagent definitions must also be listed in the project's CLAUDE.md before the coordinator can see them.

B. The tool used to spawn subagents (the Task tool) is not in the coordinator's `allowedTools`, so it cannot invoke them.

C. The coordinator needs `tool_choice` set to `any` so that it is forced to delegate rather than answer directly.

D. Subagent descriptions must be shorter than 50 characters, otherwise the coordinator ignores them.

QUESTION 5 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A user asks your research system for a report on "the impact of AI on the creative industries". The report covers only digital art and illustration. Traces show the coordinator created three subtasks: AI in digital art, AI in illustration and AI in graphic design. Every subagent did its own subtask thoroughly. Where is the root cause?

A. The search subagent's queries were too specific and should have used broader keywords.

B. The synthesis subagent dropped content about music and film when it merged the findings.

C. The coordinator's task decomposition was too narrow and missed areas such as music, writing and film.

D. The subagents ran out of context window and silently truncated their coverage.

QUESTION 6 · SCENARIO: CUSTOMER SUPPORT AGENT

Company policy says refunds over $500 require human approval. The system prompt states this clearly in bold, but audits show the agent processes about 3% of over-limit refunds itself. What is the most effective remediation?

A. Repeat the rule in capital letters at both the start and end of the system prompt.

B. Add five few-shot examples of the agent correctly declining over-limit refunds.

C. Require the agent to rate its confidence before each refund and escalate when confidence is below 0.8.

D. Intercept `process_refund` calls programmatically (for example with a pre-tool-use hook) and block or redirect any call above $500 to the escalation workflow.

QUESTION 7 · SCENARIO: CUSTOMER SUPPORT AGENT

Your agent calls tools from three MCP servers. One returns timestamps as Unix epoch seconds, one as ISO 8601 strings, and one as `MM/DD/YYYY`. The agent sometimes gets the order of events wrong when explaining a customer's history. What is the best fix?

A. Add a system-prompt instruction telling Claude to convert all dates to ISO 8601 before reasoning about them.

B. Add a post-tool-use hook that normalises every timestamp in tool results to a single format before the model sees them.

C. Ask each vendor to change their MCP server to emit ISO 8601 before going to production.

D. Remove the timestamps from tool results and ask the customer to confirm the order of events.

QUESTION 8 · SCENARIO: STRUCTURED DATA EXTRACTION

You are automating invoice processing. Every invoice goes through the same three steps: extract fields, validate them against the purchase-order system, and post to the ledger. The steps never change order. Which architecture fits best?

A. A fully autonomous agent with all ledger and PO tools that decides for itself which step to run next.

B. A coordinator with three subagents that negotiate with each other over the order of operations.

C. A single prompt that asks Claude to do all three steps and return the ledger posting.

D. A fixed workflow (prompt chain) that runs the three steps in sequence, with programmatic checks between them.

QUESTION 9 · SCENARIO: DEVELOPER PRODUCTIVITY

An engineer asks Claude to "add comprehensive tests to our legacy billing module". Nobody knows the module's structure, dependencies or which parts are risky. How should the work be decomposed?

A. As a fixed five-step prompt chain written up front: list files, write unit tests, write integration tests, run tests, report.

B. Adaptively: first map the module's structure and dependencies, identify the high-impact areas, then build a prioritised plan that is revised as new dependencies are discovered.

C. By immediately spawning one subagent per source file to write tests in parallel, with no planning phase.

D. As a single pass where Claude reads the whole module into context and writes all tests at once.

QUESTION 10 · SCENARIO: DEVELOPER PRODUCTIVITY

On Monday you spent a long Claude Code session analysing a service's architecture. Overnight, a teammate merged a large refactor that renamed and moved most of the modules. On Tuesday, what is the best way to continue?

A. Resume Monday's session with `--resume` and continue exactly where you left off.

B. Resume Monday's session and ask Claude to trust its earlier tool results because the logic is unchanged.

C. Start a new session seeded with a concise, structured summary of Monday's conclusions, and let Claude re-explore the refactored code.

D. Fork Monday's session twice so that one branch can analyse the old code and one the new code.

QUESTION 11 · SCENARIO: DEVELOPER PRODUCTIVITY

After a shared investigation of a performance problem, you want Claude to explore two competing fixes (a caching layer versus a query rewrite) without one exploration contaminating the other. Both need the same baseline analysis. What should you use?

A. Ask the same session to implement the caching approach, then revert and implement the query rewrite.

B. Open two terminals and `--resume` the same session in both at the same time.

C. Copy the analysis into CLAUDE.md and start two blank sessions that read it.

D. Fork the session, so that each approach continues from the same baseline context in its own independent branch.

QUESTION 12 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your coordinator delegates with prompts such as "Analyse these documents." Subagent outputs vary wildly in depth and format, and attribution is often lost. What is the best improvement to the coordinator's delegation prompts?

A. Give subagents detailed step-by-step procedures to follow for every possible document type.

B. State the goal, the quality criteria and the required output structure, and pass the prior findings with their source metadata (URL, document name, page) in a structured format.

C. Raise subagent temperature so they explore the documents more thoroughly.

D. Tell subagents to ask the coordinator follow-up questions whenever an instruction is unclear.

QUESTION 13 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

An architect proposes letting subagents message each other directly, without going through the coordinator, to save tokens. What is the strongest argument for keeping communication routed through the coordinator?

A. The coordinator gives you a single point for observability, consistent error handling and control over what information flows to which agent.

B. The Agent SDK does not let subagents produce output that another subagent could read.

C. Direct communication always uses more tokens than routing through a coordinator.

D. Subagents cannot run tools unless the coordinator approves each call.

QUESTION 14 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

The coordinator reviews the synthesis subagent's draft and notices that two of the five research questions have thin evidence. What should the coordinator do?

A. Send targeted follow-up queries for the thin areas to the search and analysis subagents, then re-run synthesis with the new findings.

B. Ask the synthesis subagent to fill the gaps from its own background knowledge so the report looks complete.

C. Re-run the synthesis subagent with a higher temperature to produce a more thorough draft.

D. Publish the draft as-is, because the coordinator should never re-delegate work that has already been done.

QUESTION 15 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer writes: "I need a refund for order 4471, my shipping address changed, and my blender's warranty claim was never answered." How should a well-designed support agent handle this?

A. Resolve the refund, then ask the customer to open separate tickets for the address and warranty issues.

B. Break the message into its three issues, investigate each (in parallel where possible) using the shared customer context, and reply with one unified response.

C. Escalate immediately to a human, since messages with more than one issue are too complex for the agent.

D. Run a single broad lookup on the customer and answer all three issues from whatever it returns.

QUESTION 16 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your synthesis subagent has access to all 18 tools in the system. It sometimes starts doing its own web searches instead of synthesising, but it does genuinely need to check simple facts such as dates now and then. What is the best tool design?

A. Give the synthesis subagent only synthesis-related tools plus a narrowly scoped fact-verification tool, and route any deeper research back through the coordinator.

B. Keep all 18 tools but add a prompt instruction telling it not to use the search tools.

C. Remove every verification capability so that the synthesis subagent cannot leave its role.

D. Let the synthesis subagent spawn its own research subagents whenever it finds a gap.

Domain 2: Tool Design & MCP Integration

Exam weight 18% · 11 questions

QUESTION 17 · SCENARIO: CUSTOMER SUPPORT AGENT

Your agent has two tools: `get_customer` ("Retrieves customer info") and `lookup_order` ("Retrieves order info"). When users ask about an order and give their email address, the agent often calls `get_customer` instead of `lookup_order`. What should you do first?

A. Expand both tool descriptions to say what inputs each accepts, what it returns, example queries, and when to use it instead of the other tool.

B. Add a system-prompt section with ten few-shot examples of correct tool routing.

C. Put a lightweight classifier in front of the agent that picks the tool before Claude sees the request.

D. Merge both tools into a single `lookup` tool that takes a free-text query.

QUESTION 18 · SCENARIO: CUSTOMER SUPPORT AGENT

Your MCP tool returns `"Operation failed"` for every error. The agent keeps retrying calls that failed because the user lacks permission, wasting turns. What is the best fix?

A. Return structured error results that include an error category (for example transient, validation, permission or business), an `isRetryable` flag and a human-readable explanation.

B. Raise an exception in the MCP server so that the agent loop crashes and an operator is paged.

C. Return an empty success result on any failure so the agent moves on to other work.

D. Have the MCP server retry every failure three times internally before returning `"Operation failed"`.

QUESTION 19 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

A search subagent's database query timed out, and the tool returned `[]`. The coordinator concluded there were no matching records and stated so in the final report. What design flaw caused this?

A. The tool did not distinguish an access failure from a valid empty result. It should have returned an error marked as transient instead of an empty list.

B. The coordinator should never trust empty results and should always repeat every search three times.

C. The subagent should have raised the timeout limit to 10 minutes to avoid any chance of failure.

D. The synthesis step should have removed any claim based on fewer than three search results.

QUESTION 20 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

One general-purpose agent has been given 18 tools covering search, document parsing, database access, email and calendar. Tool-selection accuracy has dropped noticeably. What is the most effective change?

A. Sort the tools alphabetically so Claude can scan the list more easily.

B. Increase `max_tokens` so that Claude has more room to reason about tool selection.

C. Set `tool_choice` to `any` so that Claude always picks some tool.

D. Split responsibilities across specialised agents and give each one only the 4-5 tools its role needs.

QUESTION 21 · SCENARIO: STRUCTURED DATA EXTRACTION

Your pipeline defines three extraction tools: `extract_invoice`, `extract_receipt` and `extract_purchase_order`. Document types are not known in advance, and you must guarantee that Claude returns structured output via one of these tools instead of plain text. Which `tool_choice` setting is right?

A. `{"type": "auto"}`

B. `{"type": "tool", "name": "extract_invoice"}`

C. `{"type": "any"}`

D. `{"type": "none"}`

QUESTION 22 · SCENARIO: STRUCTURED DATA EXTRACTION

Your pipeline must always run `extract_metadata` before any enrichment tools are called, but on the first turn Claude sometimes jumps straight to enrichment. What is the most direct fix?

A. Rename the tool to `step1_extract_metadata` so that Claude recognises it should run first.

B. Add a sentence to the system prompt saying metadata extraction should usually come first.

C. Set `tool_choice` to `any` on the first request so that a tool is always called.

D. On the first request set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}`, then handle enrichment in the following turns.

QUESTION 23 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team wants every developer's Claude Code to connect to the company's GitHub MCP server. Each developer has their own personal access token, and tokens must never be committed. How should you configure this?

A. Commit `.mcp.json` with a shared service-account token so that setup is identical for everyone.

B. Ask each developer to add the server to their own user-level config by hand, with no project file.

C. Define the server in the project-scoped `.mcp.json`, commit it, and reference the token with environment-variable expansion such as `${GITHUB_TOKEN}`.

D. Put the server URL and setup steps in CLAUDE.md so Claude can connect when it needs to.

QUESTION 24 · SCENARIO: CUSTOMER SUPPORT AGENT

Your agent makes many exploratory tool calls at the start of each session (listing knowledge-base categories, listing article IDs) just to find out what content exists before it can answer. What MCP feature best reduces this?

A. Expose the content catalogue as an MCP resource so the agent can see what data is available without making exploratory tool calls.

B. Increase the agent's tool-call limit so exploration finishes more quickly.

C. Combine all the listing tools into one tool that returns the whole knowledge base on every call.

D. Hard-code every article ID in the system prompt and update it by hand each week.

QUESTION 25 · SCENARIO: DEVELOPER PRODUCTIVITY

In Claude Code, you need to find every place in a large repository where the function `processRefund` is called. Which built-in tool is most appropriate?

A. Grep, to search file contents for the function name.

B. Glob, to match file paths against a name pattern.

C. Read, applied to every file in the repository in turn.

D. Write, to create an index file listing every function definition.

QUESTION 26 · SCENARIO: DEVELOPER PRODUCTIVITY

Claude Code's Edit tool fails because the text to replace appears three times in a file, and only one of those occurrences should change. What is the correct approach?

A. Expand the text being matched with enough surrounding lines to make it unique. If that isn't possible, read the file and write back the complete modified content.

B. Use the replace-all option, since all three occurrences are identical anyway.

C. Retry the same Edit call, because uniqueness checks sometimes fail intermittently.

D. Delete the file and ask Claude to regenerate it from memory with the change applied.

QUESTION 27 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your system has a single `analyze_document` tool that subagents use for very different purposes: pulling out data points, summarising, and checking claims. Outputs are inconsistent and hard to validate. What is the best redesign?

A. Split it into purpose-specific tools such as `extract_data_points`, `summarize_content` and `verify_claim_against_source`, each with clear input and output contracts.

B. Add a free-text `mode` parameter to `analyze_document` and describe all modes in one long description.

C. Keep one tool but post-process its output with regexes to work out which purpose was intended.

D. Remove the tool and have subagents do all document analysis in their own reasoning.

Domain 3: Claude Code Configuration & Workflows

Exam weight 20% · 12 questions

QUESTION 28 · SCENARIO: CLAUDE CODE FOR TEAMS

A new team member's Claude Code doesn't follow the team's coding conventions, though it does for the tech lead. The conventions live in the tech lead's `~/.claude/CLAUDE.md`. What is the right fix?

A. Ask the new team member to copy the tech lead's `~/.claude/CLAUDE.md` onto their machine.

B. Add the conventions to `.claude/settings.local.json` in the repository.

C. Move the conventions into the project-level `CLAUDE.md` in the repository and commit it so that every clone loads them.

D. Turn the conventions into a slash command that each developer must remember to run at the start of every session.

QUESTION 29 · SCENARIO: CLAUDE CODE FOR TEAMS

Your React codebase keeps test files next to the components they test (for example `src/cart/Button.test.tsx`) in dozens of directories. You want test-writing conventions to apply whenever Claude works on a test file, but not to load otherwise. What is the best mechanism?

A. A `CLAUDE.md` file in every directory that contains tests.

B. A rule file in `.claude/rules/` whose frontmatter sets `paths` to a glob such as `**/*.test.tsx`.

C. One section in the root `CLAUDE.md` holding all testing conventions.

D. A user-level skill that each developer invokes before writing tests.

QUESTION 30 · SCENARIO: CLAUDE CODE FOR TEAMS

You've written a `/review` command that applies your team's review checklist. Everyone on the team should get it automatically when they clone the repository. Where should it live?

A. In `.claude/commands/review.md` inside the repository.

B. In `~/.claude/commands/review.md` on the author's machine.

C. As a section of the project `CLAUDE.md` titled "/review".

D. In `.claude/settings.local.json` under a `commands` key.

QUESTION 31 · SCENARIO: CLAUDE CODE FOR TEAMS

Your team's `/dependency-audit` skill explores the codebase and produces a lot of verbose intermediate output, which clutters the main conversation and uses up context. What is the most appropriate fix?

A. Set `context: fork` in the skill's SKILL.md frontmatter so that it runs in an isolated sub-agent context and returns only its result.

B. Add an instruction to the skill telling Claude to be more concise during exploration.

C. Run `/compact` straight after every invocation of the skill.

D. Move the skill's instructions into the root CLAUDE.md so they are always loaded.

QUESTION 32 · SCENARIO: DEVELOPER PRODUCTIVITY

You need to migrate your service from one logging library to another. The change touches about 45 files and requires decisions about structured fields, log levels and context propagation. How should you start in Claude Code?

A. Use direct execution and let Claude change files one at a time as it finds them.

B. Use plan mode to explore the codebase and agree an approach before any files are changed.

C. Ask Claude to rewrite all 45 files in a single response to keep the change consistent.

D. Skip Claude Code and write a regex-based codemod by hand.

QUESTION 33 · SCENARIO: DEVELOPER PRODUCTIVITY

You describe in prose how a data-transformation function should handle edge cases. Claude's implementation keeps handling them inconsistently, even after two rounds of rewording. What is the most effective next step?

A. Rewrite the prose description a third time with more adjectives and emphasis.

B. Ask Claude to try harder and double-check its work before replying.

C. Give two or three concrete input-to-output examples that cover the edge cases.

D. Switch to plan mode and ask Claude to produce a plan for the function.

QUESTION 34 · SCENARIO: CLAUDE CODE IN CI/CD

You added a Claude Code step to a GitHub Actions workflow. The job hangs until it times out, apparently waiting for input. What is the correct fix?

A. Set the environment variable `CLAUDE_HEADLESS=true` in the job.

B. Pipe `yes` into the command so that every prompt is auto-confirmed.

C. Add `--batch` so that the request goes through the Message Batches API.

D. Run Claude Code in non-interactive print mode with the `-p` (`--print`) flag.

QUESTION 35 · SCENARIO: CLAUDE CODE IN CI/CD

Your CI job uses Claude Code to review pull requests, and you want to post each finding as an inline PR comment at the right file and line. What should you configure?

A. Ask Claude in the prompt to "format your findings so they are easy to parse" and split the text with regexes.

B. Write the findings to CLAUDE.md and have the CI job read that file afterwards.

C. Run the review interactively and have a developer copy the findings into comments.

D. Use `--output-format json` with a `--json-schema` that defines fields for file, line, severity and message, then parse the result.

QUESTION 36 · SCENARIO: CLAUDE CODE IN CI/CD

In your pipeline, Claude generates a change and then, in the same session, reviews its own work. The reviews rarely find real problems, but human reviewers often do. What is the best improvement?

A. Run the review in a separate, independent Claude instance that doesn't have the generator's reasoning in its context.

B. Tell Claude to "review extremely carefully" before it finishes the generation session.

C. Enable extended thinking in the same session so the self-review is more thorough.

D. Have the same session review the change twice and merge the two sets of findings.

QUESTION 37 · SCENARIO: CLAUDE CODE IN CI/CD

Your automated review re-runs on every push to a PR. Developers complain that it posts the same comments again after each push, including about issues they've already fixed. What should you change?

A. Only run the review on the first push to each PR.

B. Delete all previous review comments before each run and post a fresh full review.

C. Raise the severity threshold so that fewer comments are posted on each run.

D. Include the earlier review findings in the context and tell Claude to report only new issues or ones that are still unresolved.

QUESTION 38 · SCENARIO: CLAUDE CODE FOR TEAMS

In your monorepo, the root CLAUDE.md has grown to 1,400 lines covering every package's standards. Package maintainers want to own their sections, and the file is hard to maintain. What is the best restructuring?

A. Shrink the file by deleting the older standards, keeping only the 200 most recent lines.

B. Turn the whole file into a single skill so that it loads only on demand.

C. Move the file into each developer's `~/.claude/CLAUDE.md` so the repository stays clean.

D. Split the standards into topic or package files and pull them in from CLAUDE.md with `@import` references, or move them into `.claude/rules/`.

QUESTION 39 · SCENARIO: CLAUDE CODE FOR TEAMS

One developer's Claude Code consistently applies a formatting rule that nobody else's does, and nobody can find the rule in the repository. What is the fastest way to diagnose this?

A. Reinstall Claude Code on that developer's machine to clear any cached rules.

B. Run `/memory` in that developer's session to see which memory files are loaded. The rule is probably in their user-level CLAUDE.md.

C. Search the model's training data for the formatting rule.

D. Add a contradictory rule to the project CLAUDE.md and see which one wins.

Domain 4: Prompt Engineering & Structured Output

Exam weight 20% · 12 questions

QUESTION 40 · SCENARIO: CLAUDE CODE IN CI/CD

Your code-review prompt says "Be conservative and only report high-confidence findings." Developers still dismiss most findings as noise and have started ignoring the tool. What is the most effective prompt change?

- **A.** Ask Claude to assign a confidence score to each finding and report only those above 0.9.
- **B.** Define explicit criteria for what to report (for example logic bugs and security issues) and what to skip (for example minor style points and established local patterns), with concrete examples.
- **C.** Change the wording to "Be extremely conservative" and repeat it at the end of the prompt.
- **D.** Lower the temperature to 0 so that the reviewer behaves more deterministically.

QUESTION 41 · SCENARIO: CLAUDE CODE IN CI/CD

Your automated reviewer covers five categories. One of them, "naming conventions", produces 70% false positives, and developers are starting to distrust every category. What is the best immediate action?

- **A.** Keep all categories enabled and add a disclaimer to every comment saying that some findings may be wrong.
- **B.** Temporarily disable the high-false-positive category to restore trust while you improve its prompt and criteria.
- **C.** Turn off the whole reviewer until every category is below 5% false positives.
- **D.** Raise the severity of the naming category so developers take it more seriously.

QUESTION 42 · SCENARIO: STRUCTURED DATA EXTRACTION

You are extracting citations from research papers. Some use inline numeric citations, some use author-year citations, and some put references only in a bibliography. Accuracy varies widely by format. What is the most effective prompt improvement?

- **A.** Add a long paragraph describing every citation style in the abstract.
- **B.** Tell Claude to skip any paper whose citation style is not numeric.
- **C.** Raise `max_tokens` so that Claude has room to consider each citation more carefully.
- **D.** Add two to four few-shot examples that show correct extraction for each citation style, including how to reason through ambiguous cases.

QUESTION 43 · SCENARIO: STRUCTURED DATA EXTRACTION

You use tool use with a strict JSON schema to extract invoices. Schema errors have disappeared, but about 4% of outputs have line items that don't add up to the stated total. What should you add?

- **A.** A stricter JSON schema with more `required` fields and tighter types.
- **B.** A switch from tool use to a prompt that asks for JSON wrapped in a code block.
- **C.** Programmatic semantic validation (for example comparing the sum of line items with the stated total), followed by a retry that includes the specific validation error.
- **D.** A post-processing step that silently overwrites the stated total with the computed sum.

QUESTION 44 · SCENARIO: STRUCTURED DATA EXTRACTION

Your contract-extraction schema marks `termination_date` as required. Many contracts don't have one, and audits find that Claude invents plausible dates for them. What is the best fix?

A. Add a prompt instruction saying "Never make up dates" and keep the field required.

B. Default the field to the contract start date plus one year when it is missing.

C. Make `termination_date` nullable or optional, and tell Claude to return null when the document doesn't state it.

D. Reject every contract that lacks a termination date before extraction runs.

QUESTION 45 · SCENARIO: STRUCTURED DATA EXTRACTION

Your schema classifies support documents with `category` as an enum of `["billing", "technical", "account"]`. About 6% of documents fit none of these and get forced into the nearest category. What is the best schema change?

A. Remove the enum and let Claude write any category it likes as free text.

B. Add an `"other"` value to the enum, plus a free-text field that describes the category when `"other"` is chosen.

C. Add 20 more specific enum values so that every document is covered.

D. Keep the enum and drop documents that don't fit.

QUESTION 46 · SCENARIO: STRUCTURED DATA EXTRACTION

Your validation-retry loop re-prompts Claude when extraction fails. Which failure is least likely to be fixed by retrying with error feedback?

A. A date was returned as `03/04/25` when the schema expects ISO 8601.

B. A required field is empty because the source document doesn't contain that information.

C. A value was put in `billing_address` when it belonged in `shipping_address`.

D. The line items are correct but `total` was copied from the subtotal line.

QUESTION 47 · SCENARIO: CLAUDE CODE IN CI/CD

You have two workloads: (1) a pre-merge check that blocks developers until it finishes, and (2) a nightly technical-debt report. Your manager wants to move both to the Message Batches API to save 50%. What do you recommend?

A. Move both to the Batches API, since it is cheaper and usually finishes in minutes.

B. Keep both synchronous, because the Batches API doesn't support structured output.

C. Use the Batches API for the nightly report only, and keep the pre-merge check on synchronous requests.

D. Move the pre-merge check to batches and keep the nightly report synchronous, so developers save money first.

QUESTION 48 · SCENARIO: STRUCTURED DATA EXTRACTION

You submitted 10,000 documents to the Message Batches API. 312 requests failed: some for context-length errors on very long documents, some for transient reasons. What is the best way to handle the failures?

A. Resubmit all 10,000 documents so that every result comes from a consistent run.

B. Resubmit the 312 failed requests exactly as they were, without changes.

C. Use each request's `custom_id` to find the failed documents, then resubmit only those, splitting the oversized ones into chunks first.

D. Discard the failed documents, since a 3% loss is within normal tolerance.

QUESTION 49 · SCENARIO: STRUCTURED DATA EXTRACTION

Documents arrive continuously, and your SLA requires extraction results within 30 hours of arrival. You want the 50% batch discount, and batches can take up to 24 hours to process. How often should you submit batches?

A. Once a day, since the 24-hour processing window fits inside the 30-hour SLA.

B. Every 30 hours, to match the SLA exactly.

C. Never; the SLA rules out batch processing entirely.

D. At least every 6 hours (for example every 4 hours), so that waiting time plus the 24-hour processing window stays within 30 hours.

QUESTION 50 · SCENARIO: CLAUDE CODE IN CI/CD

Your reviewer analyses a 14-file pull request in a single pass. Feedback depth varies from file to file, and it sometimes contradicts itself across files. What review design is most effective?

A. Run the single-pass review three times and keep only the findings that appear in every run.

B. Require developers to split every pull request into ones of three files or fewer.

C. Review each file in its own pass for local issues, then run a separate integration pass that looks at interactions across files.

D. Use a single pass but tell Claude to give every file equal attention.

QUESTION 51 · SCENARIO: CLAUDE CODE IN CI/CD

You want to learn systematically why developers dismiss certain review findings, so you can improve the prompt. What change to the review output best supports this?

A. Add a free-text `explanation` field with no fixed structure to each finding.

B. Ask developers to write a paragraph explaining every dismissal.

C. Record only the total number of findings per PR.

D. Add a field such as `detected_pattern` to each finding that records which code pattern triggered it, so dismissals can be grouped and analysed by pattern.

Domain 5: Context Management & Reliability

Exam weight 15% · 9 questions

QUESTION 52 · SCENARIO: CUSTOMER SUPPORT AGENT

In long support conversations, the agent periodically summarises older turns to save context. After summarisation it sometimes misquotes refund amounts and order numbers. What is the best fix?

- **A.** Keep a persistent "case facts" block (IDs, amounts, dates, statuses) outside the summarised history and include it unchanged in every prompt.
- **B.** Summarise more often so that each summary covers fewer turns.
- **C.** Tell the summariser to "be precise about numbers".
- **D.** Turn summarisation off and let the context fill until the API returns an error.

QUESTION 53 · SCENARIO: CUSTOMER SUPPORT AGENT

Your `lookup_order` tool returns about 40 fields per order (warehouse codes, internal flags, audit history), but the agent only ever uses about five. Long conversations hit context limits. What is the most effective change?

- **A.** Upgrade to a model with a larger context window and keep returning all 40 fields.
- **B.** Trim tool results to the fields that are relevant before adding them to the conversation, for example in the tool itself or in a post-tool-use hook.
- **C.** Ask the agent to ignore irrelevant fields when it reads tool results.
- **D.** Drop the oldest messages once the conversation goes over a token threshold.

QUESTION 54 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Your coordinator combines findings from eight subagents into one long input for synthesis. The final report reliably covers the first and last subagents' findings but often leaves out the middle ones. What is the best mitigation?

- **A.** Reverse the order of subagent results on alternate runs so that coverage evens out over time.
- **B.** Remove the middle subagents' findings, since they are rarely used.
- **C.** Put a summary of key findings at the start of the input and organise the detailed results under clear section headers.
- **D.** Increase the synthesis model's temperature to encourage it to explore the whole input.

QUESTION 55 · SCENARIO: CUSTOMER SUPPORT AGENT

Midway through a conversation, a customer writes: "I want to talk to a human." The agent is confident it can solve the issue, a simple address change, in one more step. What should it do?

- **A.** Finish the address change first, then offer to escalate if the customer still wants to.
- **B.** Ask the customer why they want a human and try to address that concern instead.
- **C.** Run sentiment analysis, and escalate only if the customer's frustration score is high.
- **D.** Respect the request and escalate to a human straight away, passing along the case context.

QUESTION 56 · SCENARIO: CUSTOMER SUPPORT AGENT

You are designing when your support agent hands off to humans. Which approach is most reliable?

A. Escalate whenever the model's self-reported confidence is below 0.7.

B. Explicit escalation criteria (such as a customer request for a human, a case policy doesn't cover, or no progress after reasonable attempts) illustrated with few-shot examples.

C. Escalate whenever sentiment analysis finds negative customer sentiment.

D. Escalate whenever the conversation goes over ten turns.

QUESTION 57 · SCENARIO: CUSTOMER SUPPORT AGENT

A customer gives the name "Alex Chen" and the city "Austin". `get_customer` returns three matching accounts. What should the agent do?

A. Choose the account with the most recent activity, since it is most likely to be the right one.

B. Combine the three accounts' order histories so the customer can see everything.

C. Escalate to a human, because ambiguous matches can only be resolved by staff.

D. Ask the customer for an additional identifier, such as email, phone or an order number, before taking any account-specific action.

QUESTION 58 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

One of four parallel search subagents fails because its data source keeps timing out after local retries. How should the failure be handled?

A. The subagent returns an empty result so the workflow can carry on without interruption.

B. The coordinator cancels the whole research workflow and reports a generic failure.

C. The subagent returns structured error context (failure type, what it tried, any partial results, possible alternatives) to the coordinator. The coordinator decides how to recover, and the final report notes the coverage gap.

D. The subagent keeps retrying indefinitely until the source responds.

QUESTION 59 · SCENARIO: STRUCTURED DATA EXTRACTION

Your extraction pipeline is 97% accurate overall on a validation set, and leadership wants to stop human review of high-confidence outputs. What should you do first?

A. Break accuracy down by document type and by field to check that no segment performs poorly, and keep sampling high-confidence outputs for review.

B. Remove human review for all outputs above the confidence threshold, since 97% exceeds the 95% target.

C. Keep full human review for everything until accuracy reaches 100%.

D. Re-run the same validation set three times to confirm the 97% figure.

QUESTION 60 · SCENARIO: MULTI-AGENT RESEARCH SYSTEM

Two credible sources found by different subagents give different values for the same statistic (market size of $4.1B versus $5.6B). How should the synthesis step handle this?

A. Average the two values and report $4.85B as the market size.

B. Report whichever value comes from the more recent search result.

C. Report both values with their sources and dates, and note the discrepancy instead of choosing one.

D. Leave the statistic out, since conflicting data is unreliable.

Answer Key

Quick reference. Detailed explanations follow.

1	C	**11**	D	**21**	C	**31**	A	**41**	B	**51**	D
2	D	**12**	B	**22**	D	**32**	B	**42**	D	**52**	A
3	B	**13**	A	**23**	C	**33**	C	**43**	C	**53**	B
4	B	**14**	A	**24**	A	**34**	D	**44**	C	**54**	C
5	C	**15**	B	**25**	A	**35**	D	**45**	B	**55**	D
6	D	**16**	A	**26**	A	**36**	A	**46**	B	**56**	B
7	B	**17**	A	**27**	A	**37**	D	**47**	C	**57**	D
8	D	**18**	A	**28**	C	**38**	D	**48**	C	**58**	C
9	B	**19**	A	**29**	B	**39**	B	**49**	D	**59**	A
10	C	**20**	D	**30**	A	**40**	B	**50**	C	**60**	C

Answers and Explanations

Why the best answer is best, and why the other options fall short.

Domain 1: Agentic Architecture & Orchestration

Question 1 · Answer: C

Continue the loop while `stop_reason` *is* `"tool_use"`*, appending each tool result to the conversation, and stop when* `stop_reason` *is* `"end_turn"`*.*

The API tells you explicitly why the model stopped. `stop_reason == "tool_use"` means Claude is waiting for tool results; `"end_turn"` means it has finished. Parsing natural-language text (keywords or regexes) is fragile, and asking for a self-reported done flag adds another thing that can go wrong. An iteration cap is a useful safety net but makes a poor primary stop condition, because it cuts off legitimately complex cases.

Question 2 · Answer: D

Subagents do not inherit the coordinator's conversation, so the coordinator must pass the search results and their source metadata explicitly in the synthesis subagent's prompt.

Each subagent starts with a fresh, isolated context. Nothing the search subagent produced reaches the synthesis subagent unless the coordinator puts it in the prompt it writes for that subagent, ideally in a structured format that keeps each claim linked to its URL and title. Giving the synthesis agent a search tool duplicates work and blurs roles. Truncation and caching do not explain citations being missing across the board.

Question 3 · Answer: B

Have the coordinator emit all five subagent (Task) tool calls in a single response so the subagents run in parallel.

When Claude returns several tool_use blocks in one response, they can be executed concurrently, which is how parallel subagents are spawned. A single subagent covering everything loses the isolation and parallelism benefits, and its context fills up. Cutting tools does not address sequential execution. Independent processes writing to a shared file remove the coordinator's ability to check quality, handle errors and synthesise.

Question 4 · Answer: B

The tool used to spawn subagents (the Task tool) is not in the coordinator's `allowedTools`*, so it cannot invoke them.*

A coordinator delegates by calling the subagent-spawning tool. If that tool is not allowed, delegation is impossible no matter how good the subagent definitions are. CLAUDE.md is not a subagent registry. Forcing `tool_choice: any` would force some tool call, not specifically delegation. There is no 50-character limit on descriptions.

Question 5 · Answer: C

The coordinator's task decomposition was too narrow and missed areas such as music, writing and film.

Read the trace in order. The subagents executed what they were given, and nothing about music, writing or film was ever assigned. Coverage gaps that come from how the work was split belong to the coordinator. Blaming search or synthesis does not fit the evidence, because those topics were never in scope.

Question 6 · Answer: D

Intercept `process_refund` *calls programmatically (for example with a pre-tool-use hook) and block or redirect any call above $500 to the escalation workflow.*

Prompt instructions are probabilistic. For a hard business rule with financial consequences, you need deterministic enforcement in code that checks the tool call's arguments before it runs. Prompt emphasis and few-shot examples can reduce the failure rate but can't take it to zero. Self-reported confidence is poorly calibrated and doesn't relate to whether a policy applies.

Question 7 · Answer: B

Add a post-tool-use hook that normalises every timestamp in tool results to a single format before the model sees them.

A post-tool-use hook transforms results deterministically at the point they enter the agent's context, so the model always reasons over consistent data. A prompt instruction pushes error-prone conversion onto the model. Waiting on vendors may never happen. Removing timestamps throws away information the agent needs.

Question 8 · Answer: D

A fixed workflow (prompt chain) that runs the three steps in sequence, with programmatic checks between them.

When the steps are known and fixed, a workflow is simpler, cheaper, easier to test and more predictable than an agent. Agents are worth their cost when the path can't be known in advance. A single giant prompt removes the checkpoints where you validate between steps. Multi-agent negotiation adds complexity the problem doesn't need.

Question 9 · Answer: B

Adaptively: first map the module's structure and dependencies, identify the high-impact areas, then build a prioritised plan that is revised as new dependencies are discovered.

Open-ended tasks with unknown scope need decomposition that adapts as the agent learns. A fixed chain written before you understand the code tends to be wrong. One subagent per file ignores cross-file dependencies and priorities. Loading everything at once doesn't scale and leaves no room to course-correct.

Question 10 · Answer: C

Start a new session seeded with a concise, structured summary of Monday's conclusions, and let Claude re-explore the refactored code.

Resuming is good when earlier context is still valid. Here most of the earlier tool results (file paths, module contents) are stale and would mislead the model. A fresh session with a summary keeps the useful conclusions and drops the outdated details. If only a few files had changed, resuming and telling Claude exactly which files changed would be reasonable.

Question 11 · Answer: D

Fork the session, so that each approach continues from the same baseline context in its own independent branch.

Forking creates independent branches that share the history up to the fork point, which is exactly what you need for exploring alternatives side by side. Sequential exploration in one session lets the first attempt bias the second. Resuming one session concurrently in two places is not a supported way to branch. CLAUDE.md is for durable project instructions, not session scratch analysis.

Question 12 · Answer: B

State the goal, the quality criteria and the required output structure, and pass the prior findings with their source metadata (URL, document name, page) in a structured format.

Good delegation says what success looks like and gives the subagent the context it lacks, while keeping content and metadata together so attribution survives. Rigid procedures for every case are brittle and crowd out judgement. Temperature does not fix missing context. A back-and-forth clarification loop adds latency and still leaves the output format unspecified.

Question 13 · Answer: A

The coordinator gives you a single point for observability, consistent error handling and control over what information flows to which agent.

Hub-and-spoke designs keep the system debuggable and predictable: one place to log, retry, enforce quality and decide context. The other options are factually wrong. Subagents do produce output, token cost depends on the design, and subagents can use their own tools.

Question 14 · Answer: A

Send targeted follow-up queries for the thin areas to the search and analysis subagents, then re-run synthesis with the new findings.

Evaluating output and re-delegating to close specific gaps is a core coordinator responsibility in an iterative refinement loop. Filling gaps from background knowledge produces unsourced claims. Temperature doesn't create evidence. Publishing silently with gaps is worse than either refining or clearly annotating what is missing.

Question 15 · Answer: B

Break the message into its three issues, investigate each (in parallel where possible) using the shared customer context, and reply with one unified response.

Multi-concern requests should be decomposed so that each concern gets the right tools and checks, then brought back together into one coherent reply. Pushing work back to the customer is poor service. Having more than one issue is not, by itself, a reason to escalate. One generic lookup will miss issue-specific data.

Question 16 · Answer: A

Give the synthesis subagent only synthesis-related tools plus a narrowly scoped fact-verification tool, and route any deeper research back through the coordinator.

Scoping tools to each role improves tool selection and keeps agents in their lanes. A narrow verify tool covers the legitimate need without opening the door to full research. Prompt-only restrictions leak. Removing verification entirely forces more round trips. Nested spawning breaks the coordinator's control over scope and cost.

Domain 2: Tool Design & MCP Integration

Question 17 · Answer: A

Expand both tool descriptions to say what inputs each accepts, what it returns, example queries, and when to use it instead of the other tool.

Tool descriptions are the main signal Claude uses to pick a tool. Minimal, overlapping descriptions cause exactly this confusion, and fixing them is cheap and targets the root cause. Few-shot examples and classifiers add cost and complexity before the basic problem is fixed. Merging tools into a vague catch-all makes selection and validation worse.

Question 18 · Answer: A

Return structured error results that include an error category (for example transient, validation, permission or business), an `isRetryable` *flag and a human-readable explanation.*

Claude can only recover sensibly if it knows what kind of failure happened. A permission error should not be retried, while a timeout might be. Crashing the loop is heavy-handed. Returning empty success hides the failure and leads to wrong conclusions. Blind internal retries waste time on errors that can never succeed.

Question 19 · Answer: A

The tool did not distinguish an access failure from a valid empty result. It should have returned an error marked as transient instead of an empty list.

"We checked and found nothing" and "we could not check" mean very different things, and a tool must keep them apart. Returning `[]` for a timeout quietly corrupts what happens downstream. Re-running every search, or applying arbitrary limits and thresholds, treats the symptoms and adds cost.

Question 20 · Answer: D

Split responsibilities across specialised agents and give each one only the 4-5 tools its role needs.

Selection gets worse as the number of tools grows, especially when they overlap. Scoping tools by role is the standard remedy. The order of tools is not the problem, more output tokens don't sharpen the choice, and forcing some tool call doesn't make it the right one.

Question 21 · Answer: C

`{"type": "any"}`

`any` forces Claude to call one of the provided tools while still letting it choose which one fits the document. `auto` allows a plain-text reply. Forcing one named tool would push receipts and POs through the invoice schema. `none` stops tool use entirely.

Question 22 · Answer: D

On the first request set `tool_choice` *to* `{"type": "tool", "name": "extract_metadata"}`*, then handle enrichment in the following turns.*

Forcing a specific tool guarantees that the required first step runs, every time. Naming conventions and prompt wording only nudge the model. `any` guarantees a tool call but not which tool.

Question 23 · Answer: C

Define the server in the project-scoped `.mcp.json`*, commit it, and reference the token with environment-variable expansion such as* `${GITHUB_TOKEN}`*.*

Project-scoped `.mcp.json` shares the configuration through version control, and environment-variable expansion keeps each person's secret out of the repo. A committed token is a security incident. Manual per-user setup loses the shared, consistent configuration. CLAUDE.md is guidance for Claude, not a way to configure MCP connections.

Question 24 · Answer: A

Expose the content catalogue as an MCP resource so the agent can see what data is available without making exploratory tool calls.

MCP resources are designed to expose readable context such as catalogues, schemas or document indexes, so the agent knows what exists up front. Raising limits doesn't remove the waste. Returning everything on every call bloats the context. Hard-coding IDs goes stale and doesn't scale.

Question 25 · Answer: A

Grep, to search file contents for the function name.

Grep searches file contents, which is what you need to find call sites. Glob matches file paths and names (for example `**/*.test.tsx`), not what is inside files. Reading every file wastes context. Writing an index is unnecessary.

Question 26 · Answer: A

Expand the text being matched with enough surrounding lines to make it unique. If that isn't possible, read the file and write back the complete modified content.

Edit needs a unique anchor. Adding context makes the target unambiguous, and Read plus Write is the fallback when targeted edits can't be anchored. Replace-all would change occurrences that must stay as they are. Retrying won't change the outcome. Regenerating from memory risks losing content.

Question 27 · Answer: A

Split it into purpose-specific tools such as `extract_data_points`*,* `summarize_content` *and* `verify_claim_against_source`*, each with clear input and output contracts.*

Specific tools with explicit contracts make selection clearer and outputs checkable. A free-text mode parameter keeps the ambiguity. Regex post-processing is fragile. Removing the tool gives up structure and the ability to validate.

Domain 3: Claude Code Configuration & Workflows

Question 28 · Answer: C

Move the conventions into the project-level `CLAUDE.md` *in the repository and commit it so that every clone loads them.*

User-level memory applies only to that user. Team conventions belong in the project-level CLAUDE.md under version control. Copying files by hand drifts over time. `settings.local.json` is for personal, uncommitted settings, not instructions. A command that has to be remembered isn't always-on guidance.

Question 29 · Answer: B

A rule file in `.claude/rules/` *whose frontmatter sets* `paths` *to a glob such as* `**/*.test.tsx`*.*

Path-scoped rules load only when matching files are involved, wherever those files live, which suits conventions spread across many directories. Per-directory CLAUDE.md files are duplicated and hard to maintain. A root section loads in every session, even when it isn't relevant. A skill relies on people remembering to invoke it.

Question 30 · Answer: A

In `.claude/commands/review.md` *inside the repository.*

Project-scoped commands in `.claude/commands/` are version-controlled and shared with every clone. The `~/.claude/commands/` directory is personal. CLAUDE.md content doesn't create a slash command. `settings.local.json` is personal and not committed.

Question 31 · Answer: A

Set `context: fork` *in the skill's SKILL.md frontmatter so that it runs in an isolated sub-agent context and returns only its result.*

Forked context isolates the skill's exploration from the main session, so only the final result comes back. Asking for brevity only partly helps. Compacting afterwards loses detail and depends on someone remembering to do it. Putting the skill in CLAUDE.md makes it load every session and doesn't isolate anything.

Question 32 · Answer: B

Use plan mode to explore the codebase and agree an approach before any files are changed.

Large, multi-file changes with design decisions benefit from plan mode, which lets you investigate safely and agree on an approach before committing to changes. Direct execution suits well-defined, small fixes. A single huge response is risky and hard to review. A codemod can't easily handle the judgement calls involved.

Question 33 · Answer: C

Give two or three concrete input-to-output examples that cover the edge cases.

When prose is being read inconsistently, concrete examples remove the ambiguity, because they show exactly what each edge case should produce. More emphasis and generic "try harder" requests rarely fix a misunderstanding. Plan mode is about exploring and designing multi-step work, not clarifying a function's expected behaviour.

Question 34 · Answer: D

Run Claude Code in non-interactive print mode with the `-p` *(*`--print`*) flag.*

`-p` runs a single prompt non-interactively and exits, which is what CI needs. The environment variable and `--batch` flag shown are not how Claude Code runs headless. Piping `yes` is a hack that doesn't address interactive mode.

Question 35 · Answer: D

Use `--output-format json` *with a* `--json-schema` *that defines fields for file, line, severity and message, then parse the result.*

Structured JSON output that follows a schema is the reliable way to pass results to automation. Free-text parsing is brittle. CLAUDE.md is for instructions, not output. A manual step defeats the purpose of CI.

Question 36 · Answer: A

Run the review in a separate, independent Claude instance that doesn't have the generator's reasoning in its context.

A model reviewing its own work in the same context tends to accept its earlier reasoning. An independent reviewer, without that context, is far more likely to catch issues. More emphasis, more thinking or repeated passes in the same context don't remove that bias.

Question 37 · Answer: D

Include the earlier review findings in the context and tell Claude to report only new issues or ones that are still unresolved.

Giving the reviewer the earlier findings lets it work incrementally: skip what has been fixed, keep what is still open, and add what is new. Reviewing only the first push misses later problems. Deleting and reposting adds noise. A higher threshold hides real issues without solving duplication.

Question 38 · Answer: D

Split the standards into topic or package files and pull them in from CLAUDE.md with `@import` references, or move them into `.claude/rules/`.

Imports and the rules directory let you keep standards modular, assign ownership and keep them focused, while still loading them when they are relevant. Deleting content loses standards. A skill fits on-demand procedures, not always-applicable standards. User-level files aren't shared.

Question 39 · Answer: B

Run `/memory` in that developer's session to see which memory files are loaded. The rule is probably in their user-level CLAUDE.md.

The `/memory` command shows which memory files are loaded, which quickly surfaces user-level or local files that aren't in version control. Reinstalling doesn't touch user memory files. Training data has nothing to do with it. Adding contradictory rules creates confusion instead of diagnosing the cause.

Domain 4: Prompt Engineering & Structured Output

Question 40 · Answer: B

Define explicit criteria for what to report (for example logic bugs and security issues) and what to skip (for example minor style points and established local patterns), with concrete examples.

Vague words like "conservative" don't tell the model where the line is. Explicit categories with examples do. Self-reported confidence is poorly calibrated. Stronger wording is still vague. Temperature affects randomness, not judgement about what counts as worth reporting.

Question 41 · Answer: B

Temporarily disable the high-false-positive category to restore trust while you improve its prompt and criteria.

Noise in one category erodes trust in all of them. Pausing just the noisy category keeps the valuable ones working while you fix it. Disclaimers don't restore trust. Turning everything off throws away real value. Raising the severity of noise makes the problem worse.

Question 42 · Answer: D

Add two to four few-shot examples that show correct extraction for each citation style, including how to reason through ambiguous cases.

Few-shot examples are the most effective way to teach consistent handling of varied formats, especially when they show the reasoning on edge cases. Abstract descriptions are less effective. Skipping papers loses data. Token limits are not what is causing the inconsistency.

Question 43 · Answer: C

Programmatic semantic validation (for example comparing the sum of line items with the stated total), followed by a retry that includes the specific validation error.

Schemas guarantee structure, not meaning. Checks like whether items sum to the total need application-level validation, and feeding the specific error back lets the model correct itself. Stricter schemas can't express arithmetic relationships. Prompt-only JSON goes backwards on reliability. Silently overwriting hides real discrepancies that may need a human.

Question 44 · Answer: C

Make `termination_date` *nullable or optional, and tell Claude to return null when the document doesn't state it.*

A required field forces the model to produce something, which encourages fabrication. Allowing null gives it an honest way to say the information isn't there. A prompt warning fights against the schema. A made-up default is still fabricated data. Rejecting contracts loses everything else in them.

Question 45 · Answer: B

Add an `"other"` *value to the enum, plus a free-text field that describes the category when* `"other"` *is chosen.*

An `other` option with a detail field keeps most outputs structured while capturing genuine exceptions, and the details show you which categories to add later. Free text loses consistency. A long list of guessed values still won't cover everything and makes classification harder. Dropping documents loses data.

Question 46 · Answer: B

A required field is empty because the source document doesn't contain that information.

Retrying works when the information is in the document and the model got the format, placement or source line wrong; feedback lets it correct those. If the information isn't in the document at all, no number of retries can produce it truthfully. Recognise that case and route it as missing data.

Question 47 · Answer: C

Use the Batches API for the nightly report only, and keep the pre-merge check on synchronous requests.

Batch processing can take up to 24 hours and has no latency guarantee, which is fine for nightly reports and unacceptable for checks developers are waiting on. "Usually fast" isn't a guarantee. Batches do support the same request features such as tools. The last option is the wrong way round.

Question 48 · Answer: C

Use each request's `custom_id` *to find the failed documents, then resubmit only those, splitting the oversized ones into chunks first.*

`custom_id` exists so you can match results back to inputs and handle failures one by one. Re-running everything wastes money. Resubmitting oversized documents unchanged will fail again. Discarding documents silently loses data.

Question 49 · Answer: D

At least every 6 hours (for example every 4 hours), so that waiting time plus the 24-hour processing window stays within 30 hours.

In the worst case a document waits for the next submission and then takes the full 24 hours to process. Submitting at least every 6 hours keeps that worst case within 30 hours. Daily submission can reach 48 hours. Batches remain viable once you size the submission interval correctly.

Question 50 · Answer: C

Review each file in its own pass for local issues, then run a separate integration pass that looks at interactions across files.

Splitting the review into focused passes gives each file consistent attention, and a dedicated integration pass catches problems that cross file boundaries. Keeping only findings that appear in all three runs throws away real issues that show up inconsistently. Forcing tiny PRs isn't always practical. Asking for equal attention doesn't fix context dilution.

Question 51 · Answer: D

Add a field such as `detected_pattern` *to each finding that records which code pattern triggered it, so dismissals can be grouped and analysed by pattern.*

Recording what triggered each finding as a structured field lets you analyse false positives by pattern and fix the criteria that cause them. Unstructured text and developer essays are hard to aggregate. Counts alone don't tell you what to fix.

Domain 5: Context Management & Reliability

Question 52 · Answer: A

Keep a persistent "case facts" block (IDs, amounts, dates, statuses) outside the summarised history and include it unchanged in every prompt.

Summaries are lossy, and exact transactional facts are what gets lost. Pulling those facts into a structured block that is never summarised keeps them exact. More frequent summarising multiplies the losses. A precision instruction helps only a little. Running out of context is not a strategy.

Question 53 · Answer: B

Trim tool results to the fields that are relevant before adding them to the conversation, for example in the tool itself or in a post-tool-use hook.

Every unused field takes up context and dilutes attention. Filtering at the source fixes the problem. A larger window delays it. Ignoring fields still costs tokens. Dropping old messages can lose critical facts.

Question 54 · Answer: C

Put a summary of key findings at the start of the input and organise the detailed results under clear section headers.

Long inputs suffer from a "lost in the middle" effect. Putting key points up front and giving the rest a clear structure helps the model attend to everything. Alternating the order just moves the gap. Removing findings loses information. Temperature doesn't fix where the model's attention goes.

Question 55 · Answer: D

Respect the request and escalate to a human straight away, passing along the case context.

An explicit request for a human is a clear escalation trigger and should be honoured without delay. Overriding it, questioning it or putting it behind a sentiment score damages trust. (This differs from a customer who is only frustrated: there, acknowledging the frustration and offering a fix first can be appropriate.)

Question 56 · Answer: B

Explicit escalation criteria (such as a customer request for a human, a case policy doesn't cover, or no progress after reasonable attempts) illustrated with few-shot examples.

Clear criteria tied to real reasons for handing off, with examples, give consistent and defensible behaviour. Self-reported confidence is poorly calibrated: the agent may be confident on hard cases and unsure on easy ones. Sentiment doesn't tell you how complex the case is. Turn counts are an arbitrary proxy.

Question 57 · Answer: D

Ask the customer for an additional identifier, such as email, phone or an order number, before taking any account-specific action.

Acting on the wrong account is a privacy and correctness failure, and the customer can easily clear up the ambiguity. Guessing by heuristic is risky. Combining accounts exposes other people's data. Escalating is unnecessary when one clarifying question solves it.

Question 58 · Answer: C

The subagent returns structured error context (failure type, what it tried, any partial results, possible alternatives) to the coordinator. The coordinator decides how to recover, and the final report notes the coverage gap.

Structured error propagation lets the coordinator make an informed choice: try another source, proceed with partial data, or flag the gap. Presenting a failure as an empty result produces false conclusions. Cancelling everything wastes the three successful results. Retrying forever isn't recovery.

Question 59 · Answer: A

Break accuracy down by document type and by field to check that no segment performs poorly, and keep sampling high-confidence outputs for review.

A strong overall number can hide weak segments, such as one document type at 70%. Checking by segment and continuing to sample high-confidence outputs make automation safe. Removing review straight away risks systematic errors. Requiring 100% is unrealistic. Re-running the same set doesn't uncover anything new.

Question 60 · Answer: C

Report both values with their sources and dates, and note the discrepancy instead of choosing one.

Keeping provenance and showing the conflict lets readers judge for themselves; the difference may reflect different years or definitions. Averaging produces a number that neither source supports. Picking one hides a real disagreement. Leaving it out loses useful information.

Score Sheet

Fill in after marking. The real exam reportedly uses a scaled score with 720/1000 to pass; 75%+ in every domain on this set is a good sign you're ready.

Domain	Questions	Correct	%	Topics to revisit
D1. Agentic Architecture & Orchestration	16			
D2. Tool Design & MCP Integration	11			
D3. Claude Code Configuration & Workflows	12			
D4. Prompt Engineering & Structured Output	12			
D5. Context Management & Reliability	9			
Total	**60**			